LLM

ASSIGNMENT 5

Task 2: Autoregressive Decoder-Only Transformer for Causal Language Modeling

Implementation, Experimental Results, and Analysis

Abstract

This task implements a GPT-style decoder-only Transformer for character-level next-token prediction on Shakespearean play text. The model uses learned character embeddings, learned positional embeddings, manually constructed multi-head scaled dot-product self-attention, a causal triangular mask, residual connections, layer normalization, and a feed-forward network. Training minimizes cross-entropy over all positions in each 128-character context window. Validation perplexity is computed as the exponential of the validation cross-entropy loss.

The executed notebook reports a character vocabulary of 77 symbols and trains one configuration: four Transformer blocks, four attention heads, and a model dimension of 128 with learned positional embeddings. Across 15 epochs, training loss decreases from 2.3273 to 1.3266. The lowest recorded validation cross-entropy is 1.4240 at epoch 14, corresponding to perplexity 4.1536. The final epoch validation loss is 1.4243 and perplexity is 4.1548.

Scope of the reported experiment

The uploaded notebook contains outputs for one baseline configuration only. It does not contain executed results for the other head counts, model dimensions, positional-encoding alternatives, or the L = 6 and L = 8 depth-scaling study. Therefore, the numerical comparison and conclusions in this report are restricted to the completed baseline run. No unexecuted results have been estimated or fabricated.

1. Objective

The objective is to model the probability of the next character given the preceding context and to generate text autoregressively from a user-supplied prefix. At each position, the decoder predicts the next character while preventing the representation at that position from attending to future positions. This is achieved using a lower-triangular causal attention mask.

2. Dataset and preprocessing

The executed notebook loads Shakespeare_data.csv and extracts the non-null values in the PlayerLine column. The lines are concatenated in their existing order with newline characters between them to form one continuous text corpus. The notebook output records a corpus length of 4,366,287 characters. The dataset is the Shakespeare plays dataset specified in the assignment.

A character-level vocabulary is constructed by taking the sorted set of unique characters in the corpus. The resulting vocabulary size is V = 77, which falls within the target range of approximately 65-80. Each character is mapped to an integer index and can be converted back to text using the reverse mapping.

The corpus is divided sequentially: the first 90% of characters are used for training and the final 10% for validation. This produces 3,929,658 training characters and 436,629 validation characters. The notebook does not create a separate held-out test split.

The notebook reads the CSV from a local runtime path named Shakespeare_data.csv. Thus, this particular execution used a file available in the Colab runtime rather than downloading it from the Kaggle URL inside the notebook. For reproducibility, the CSV must be available at that path when the notebook is rerun.

3. Autoregressive sequence formulation

The context length is fixed at T = 128. For a sequence of characters, an input window and its target are offset by one position:

Input: x = [x1, x2, ..., x128]

Target: y = [x2, x3, ..., x129]

The dataset advances by 128 characters between windows. A batch contains 64 windows. The training DataLoader shuffles windows and drops an incomplete final batch; the validation DataLoader preserves window order and also drops an incomplete batch. The executed run uses 479 training batches and 53 validation batches per epoch.

4. Model architecture

The decoder-only model consists of a character embedding layer, a positional embedding layer, four Transformer blocks, a final layer normalization, and a linear language-model output head. The configuration used in the run is summarized below.

Parameter	Executed value
Vocabulary size V	77 characters
Context length T	128
Transformer depth L	4 blocks
Attention heads h	4
Model dimension d_model	128
Dimension per head d_k	32
Feed-forward dimension	512 (4 x d_model)
Positional encoding	Learned 1D embedding
Optimizer / learning rate	AdamW / 0.001
Batch size / maximum epochs	64 / 15

4.1 Multi-head scaled dot-product attention

The query, key, and value tensors are obtained from separate learned linear projections. Each projected tensor is reshaped into four heads, each of dimension 32. The implementation computes attention scores using matrix multiplication and scales them by the square root of the head dimension:

$$Attention(Q, K, V) = softmax((Q\ K^T) / sqrt(d_k) + M_causal)\ V$$

The four head outputs are transposed, concatenated along the feature dimension, and passed through an output projection. The code uses basic tensor operations (linear projections, reshape, transpose, matrix multiplication, softmax, and masking) rather than a built-in multi-head attention layer.

4.2 Causal masking

A lower-triangular mask is registered for the maximum context length. For query position i and key position j, entries with j <= i are retained, while entries with j > i are replaced with negative infinity before softmax. Consequently, each position can attend to itself and earlier characters, but cannot attend to future characters. The mask is broadcast across the batch and attention heads.

4.3 Transformer block

Each of the four blocks uses a pre-layer-normalization structure. The input first passes through layer normalization and multi-head causal attention, followed by a residual addition. The result then passes through a second layer normalization and a two-layer feed-forward network with ReLU activation and hidden dimension 512, followed by another residual addition. After the final block, a layer normalization and linear projection produce a vocabulary-sized logit vector at every position.

4.4 Positional encoding

The executed model uses learned positional embeddings. The notebook also defines a fixed sinusoidal positional encoding option, but the reported training cell instantiates the model with pe_mode="learned". No sinusoidal run is included in the saved outputs, so the two methods cannot be compared empirically from this execution.

5. Training objective and evaluation

For every batch, the model produces logits with shape [B, T, V]. The logits are reshaped and cross-entropy is calculated against the shifted target characters at all sequence positions. AdamW updates the model parameters using a learning rate of 0.001. Validation loss is calculated without gradient updates. Validation perplexity is defined as:

$$PPL = exp(L_validation)$$

The convergence rule specified in the assignment is |L_bar(k) - L_bar(k-1)| < 10^-4, where L_bar(k) is the average training loss in epoch k. The notebook checks this difference after each epoch and stops early if the threshold is met, subject to a maximum of 15 epochs.

6. Results

The model was trained on the CUDA device, as printed by the notebook. All 15 epochs completed. The training and validation metrics recorded at the end of each epoch are shown below.

Epoch	Train CE	Validation CE	Validation PPL	Train-loss change
1	2.3273	1.9743	7.2014	-
2	1.8175	1.7174	5.5699	0.5098
3	1.6412	1.6147	5.0261	0.1763
4	1.5527	1.5591	4.7547	0.0885
5	1.4980	1.5224	4.5831	0.0547
6	1.4593	1.4979	4.4721	0.0387
7	1.4304	1.4772	4.3808	0.0289
8	1.4087	1.4716	4.3564	0.0217
9	1.3902	1.4529	4.2753	0.0185
10	1.3755	1.4483	4.2559	0.0147
11	1.3631	1.4429	4.2328	0.0124
12	1.3525	1.4344	4.1970	0.0106
13	1.3429	1.4289	4.1742	0.0096
14	1.3343	1.4240	4.1536	0.0086
15	1.3266	1.4243	4.1548	0.0077

6.1 Training and validation loss

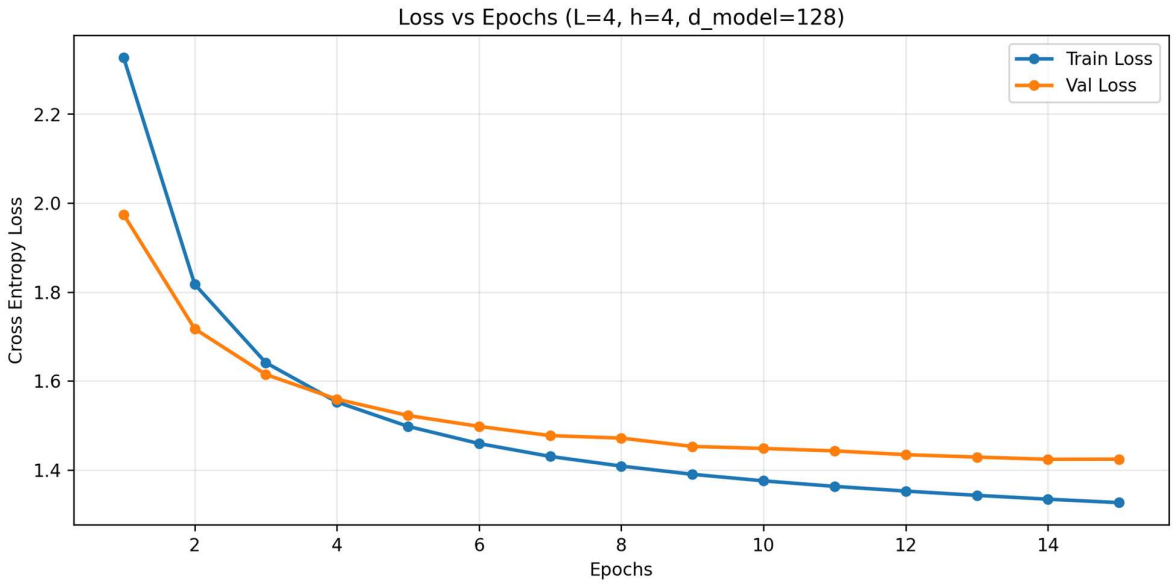


Figure 1. Training and validation cross-entropy across the 15 completed epochs. The training curve continues to decrease, while validation loss approaches a plateau near the end of the run.

6.2 Validation perplexity

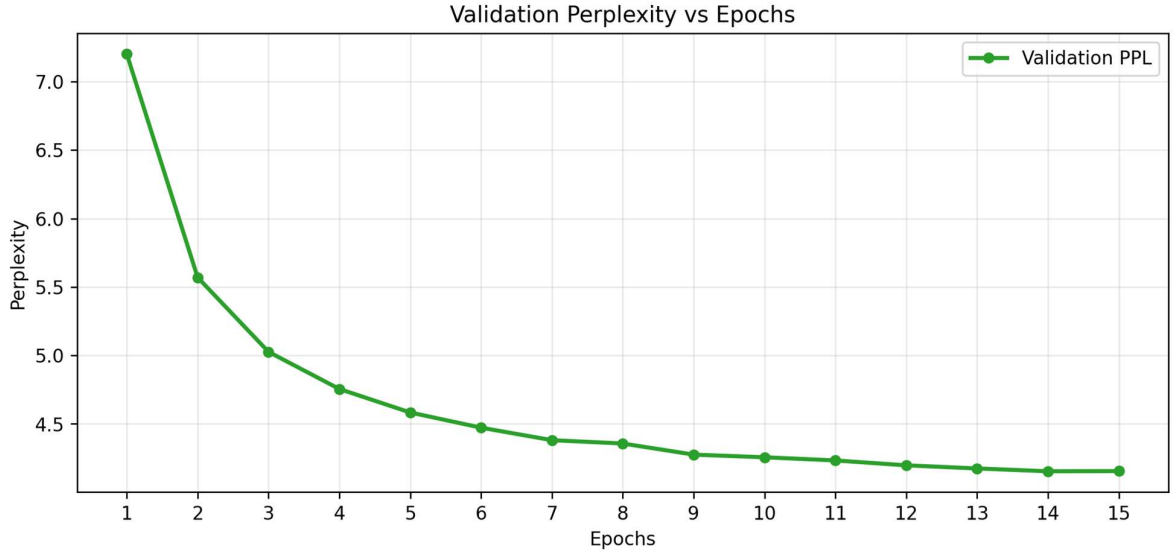


Figure 2. Validation perplexity by epoch. The minimum observed value is 4.1536 at epoch 14.

Across 15 epochs, average training loss decreased from 2.3273 to 1.3266. Validation cross-entropy decreased from 1.9743 to 1.4243. The lowest validation loss was 1.4240 in epoch 14, with perplexity 4.1536; epoch 15 ended slightly higher at 1.4243 and 4.1548.

6.3 Convergence check

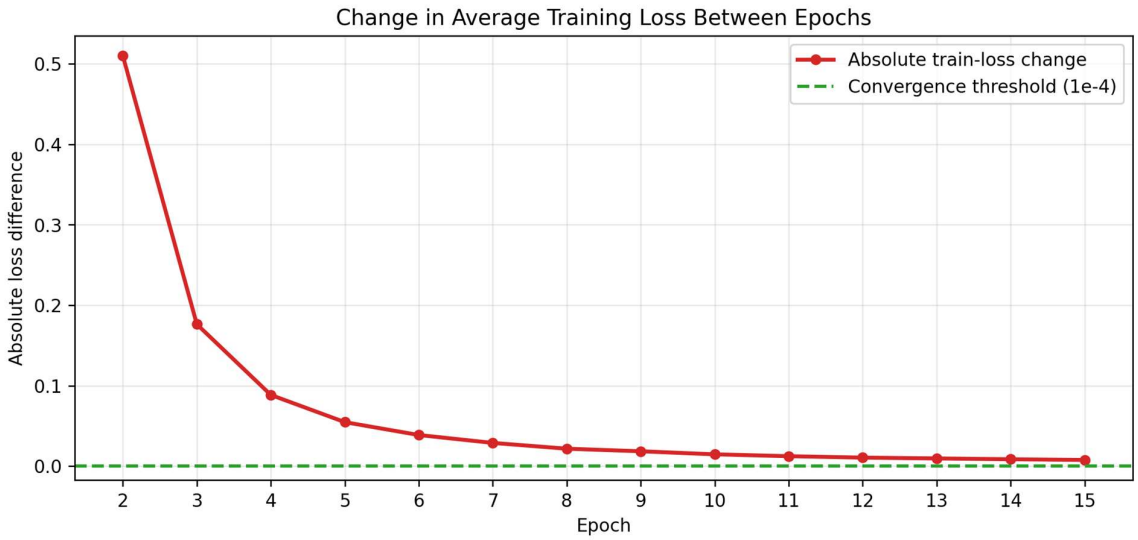


Figure 3. Absolute difference between consecutive epoch-average training losses. The final difference, from epoch 14 to epoch 15, is 0.0077, which remains larger than the required threshold of 0.0001. The stopping criterion was therefore not satisfied within the 15-epoch training budget.

6.4 Main quantitative findings

Metric	Observed result	Interpretation
Initial training CE	2.3273 (epoch 1)	Starting point for optimization
Final training CE	1.3266 (epoch 15)	Training objective continued to improve
Best validation CE	1.4240 (epoch 14)	Lowest validation loss in the recorded run
Best validation PPL	4.1536 (epoch 14)	Lowest validation perplexity in the recorded run
Final validation PPL	4.1548 (epoch 15)	Marginally above the epoch-14 minimum
Convergence condition	Not reached	Final loss change 0.0077 > 0.0001

No independent test metric is reported because the notebook creates a training split and a validation split only. The assignment deliverable for Task 2 specifically requests validation cross-entropy and perplexity, which are reported here.

7. Autoregressive text generation

Generation begins with an encoded prompt prefix. At each step, the model evaluates the most recent 128 characters (or the entire available prefix if shorter), takes the logits at the final position, and samples the next character. The new character is appended to the context and the process repeats for 50 new characters.

For greedy decoding, the character with the largest logit is selected. For temperature-scaled sampling, logits are divided by temperature before softmax and the next character is sampled from the resulting categorical distribution. Lower temperatures concentrate probability on higher-ranked characters; higher temperatures flatten the distribution and increase variation.

7.1 Sample generations from the executed notebook

The notebook generated 20 samples from five prefixes using greedy decoding and temperatures 0.5, 0.8, and 1.2. Five representative samples are reproduced below exactly as recorded in the notebook output.

Prompt	Decoding	Generated continuation (prompt included)
ROMEO:	Greedy	ROMEO: The thing of the court of the court, and the thing
KING:	Temperature = 0.5	KING: Why, who is the place of court, and so many That t
What light	Temperature = 0.8	What light that beat an advice and news? Her long, sir, unlac
To be or	Greedy	To be or the part of the court, The sun of the state of the
Wherefore art	Temperature = 1.2	Wherefore art our drinp? My lord: granted Helence, sir! I shall

7.2 Qualitative observations

The greedy sample repeats phrases such as “the court” and “the streets”, illustrating that repeatedly choosing the most likely next character can produce repetitive continuations. At temperature 0.5, the sampled text tends to remain closer to high-probability character sequences, although it can still contain awkward or incomplete phrases. Temperature 0.8 produces more variation while retaining some recognizable dramatic phrasing. At temperature 1.2, the output is more varied but contains visibly less plausible spellings and word fragments, for example “drinp” and “Helence”.

These observations are qualitative and are based on a small number of generated samples. They do not establish a general quantitative ranking of decoding temperatures.

8. Causal attention visualization

The notebook visualizes attention weights from the first Transformer block for the 15-character input string “O Romeo, Romeo!”. The figure contains one panel per attention head and preserves the original colour heatmaps generated by the executed notebook.

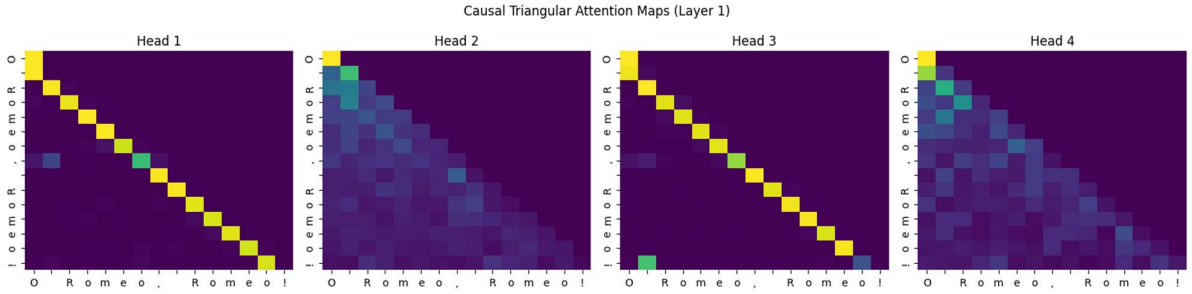


Figure 4. First-layer causal attention maps for four attention heads. Rows correspond to query positions and columns to key positions. The lower-triangular structure reflects the causal restriction: positions do not assign attention to future characters. Differences among panels show that the heads learn different attention-weight patterns.

9. Discussion

9.1 Learning behaviour

The training curve falls quickly during the first few epochs and then flattens. Training cross-entropy falls by 1.0007 between epoch 1 and epoch 15, a reduction of approximately 43.0% relative to its first-epoch value. Validation cross-entropy falls by 0.5500, approximately 27.9% relative to epoch 1. Validation perplexity decreases from 7.2014 to 4.1548 by the final epoch, approximately a 42.3% reduction.

The validation loss reaches its minimum at epoch 14 and increases by 0.0003 in epoch 15, while training loss continues to decrease. This small divergence is consistent with a modest generalization gap near the end of training. The observed difference is small, and the available run is not sufficient to conclude severe overfitting.

9.2 Effect of attention head count h

Only h = 4 was trained in the saved experiment. The assignment also asks for h = 2, but no h = 2 result is available. Therefore, the effect of changing the number of heads cannot be inferred empirically from this notebook output. In the implemented model, d_model = 128 and h = 4 give d_k = 32 per head. A valid head-count study would keep the remaining settings and data protocol fixed, train an otherwise identical h = 2 model, and compare validation loss, perplexity, and runtime.

9.3 Effect of Transformer depth L

The saved run uses L = 4. No L = 2 baseline and no L = 6 or L = 8 depth-scaling runs are included. As a result, this report cannot make a data-supported claim about how depth affects validation performance, convergence speed, training stability, or compute time. The bonus depth-scaling study remains incomplete in the provided outputs.

9.4 Positional encoding

The learned positional embedding option was used during training. Although the source code defines sinusoidal positional encodings, no trained sinusoidal model or comparative metric is present. The effect of learned versus sinusoidal positional encoding is therefore not measured here.

10. Implementation observations and limitations

The implementation satisfies the core decoder-only mechanics in the executed configuration: Q, K, and V are projected separately; heads are split and concatenated with tensor operations; scaled dot-product attention is computed explicitly; and a lower-triangular mask blocks future positions. The model predicts a vocabulary distribution at each of the 128 context positions and is trained against the one-character-shifted target.

The experimental evidence has the following limits:

- Only one hyperparameter configuration was executed and saved: L = 4, h = 4, d_model = 128, learned positional embeddings.
- The notebook uses a 90/10 train-validation split and does not report an independent test split.
- The stopping threshold of 10^-4 was not reached within 15 epochs.
- The notebook does not record elapsed time per epoch, so runtime comparisons are unavailable.
- The model was not saved as a checkpoint in the supplied notebook outputs; generation and attention plots were produced using the in-memory model after epoch 15.
- The dataset is read from a local CSV path in the executed notebook; the file must be present to reproduce the run.

11. Conclusion

A character-level decoder-only Transformer was implemented and trained on Shakespeare play dialogue. The executed four-layer, four-head model with d_model = 128 and learned positional embeddings achieved a minimum validation cross-entropy of 1.4240 and validation perplexity of 4.1536 at epoch 14. The loss curves show substantial improvement during training followed by a gradual plateau. The generation samples illustrate how temperature changes the diversity and plausibility of character-level continuations, while the attention visualization displays the expected causal triangular structure.

The core causal language-modeling objective and generation workflow were demonstrated. However, the full experimental comparison requested in the assignment - including h = 2, alternate d_model values, sinusoidal positions, L = 2, and the L = 6 / L = 8 depth study - is not supported by the uploaded notebook outputs and should be run before making comparative claims.

Appendix: Reproducibility details

Setting	Recorded value
Data column	PlayerLine
Corpus length	4,366,287 characters
Vocabulary	77 characters
Train/validation split	First 90% / final 10% of the concatenated stream
Window length	128 characters
Batch size	64
Optimizer	AdamW, learning rate 0.001
Device	CUDA
Training epochs	15
Generation length	50 new characters per sample
Attention visualization	First Transformer block, all four heads, prompt “O Romeo, Romeo!”